

Coded Prototype Development Plan for Blithob Professionals (Phase 1)

Overview

This plan outlines the steps required to build a fully coded prototype of the Blithob Professionals job management platform. Unlike a visual-only prototype, this version will be implemented as a working front-end with minimal back-end functionality, providing realistic interactions and a structure that can later be expanded into the full production application. The goal is to deliver an interactive product that the client can click through and that reflects the intended visual and functional design.

Guiding Principles

- **Production-grade code:** Build the front-end in a modern framework (e.g., React with TypeScript) using modular components and a clear file structure, so it can be extended for production.
- **No back-end integration:** The prototype should not connect to a real database. Use placeholder data (hard-coded arrays or JSON files) and mock functions to simulate API responses. This ensures the front-end is functional without relying on a back-end and keeps the focus on the user interface and flows. All back-end plans are documented separately in the full project task list.
- **Brand adherence:** Incorporate the color palette, typography and design patterns defined in the design plan (e.g., primary blue, secondary navy, orange accent, Montserrat/Poppins fonts).
- **Plain language:** Use simple words and labels throughout the user interface ¹.
- **Focus on core flows:** Implement the main navigation and pages for Admin, Worker and Trainer roles, ensuring each flow is functional and intuitive.
- **Neutral colour scheme:** Design the app with white and off-white backgrounds and use brand colours sparingly (e.g., for buttons or active icons). This creates a clean, modern feel akin to iOS 2026 and ensures the interface doesn't feel cluttered or overly branded. Save heavier brand colours for the marketing landing page.

Phase Breakdown and Tasks

1. Initial Setup

1. **Create project repository** – Initialize a new Git repository. Set up separate folders for `src`, `components`, `pages` and `styles`.
2. **Set up tooling** – Install Node.js, create a `package.json`, install dependencies (e.g., `React`, `TypeScript`, `Tailwind CSS` or another styling framework). Configure ESLint and Prettier for code linting.

3. **Define mock data structure** – Decide how to represent users, jobs, training tracks and payouts as JavaScript objects or JSON files. Create sample data to use throughout the prototype.

2. Base Layout & Routing

1. **Define routes** – Use a routing library (e.g., React Router) to define paths for: `Landing`, `Login`, `AdminDashboard`, `JobList`, `JobDetails`, `CreateJob`, `WorkerDashboard`, `Training`, `TrainerDashboard`, `SubmissionReview`, `Payouts`.
2. **Create layout components** – Implement a main layout with a header, sidebar or navigation bar, and content area. Include responsive styles for desktop and tablets.
3. **Add a global theme** – Configure Tailwind or CSS variables with the color palette and font family; create utility classes for consistent spacing, typography and colors.

3. Landing Page Placeholder

1. **Create hero section** – Implement a full-width section with a headline, subheadline and two call-to-action buttons (“Apply as a Remote Worker”, “Learn about Visa Services”). Use a background image with a blue overlay.
2. **Add services overview** – Represent Travel, Visa, Recruitment and Remote Jobs as cards with icons and short descriptions. Apply hover effects.
3. **Build login/signup CTA** – Add a login button or link to the `Login` route. This page should not include the full content; it should serve as a placeholder for the final marketing site.

4. Authentication Flow

1. **Create role selection page** – Replace traditional login and sign-up forms with a simple role selection screen. Provide two large buttons: “Sign in as Admin” and “Sign in as Worker”. Clicking a button sets the current user role in application state (e.g., React Context) and navigates to the appropriate dashboard.
2. **Simplified sign-up** – For worker onboarding, display a short form after role selection to collect basic details (name, email) and store them in local state or local storage. This does not persist data beyond the session but allows demonstration of the flow.
3. **Session management** – Store the selected role and user details in state management or local storage to persist the session across page reloads. Add a “Sign Out” option to clear the session.

5. Admin Dashboard and Job Management

1. **Dashboard metrics** – Create components for metrics cards (Workers in training, Workers ready, Active Jobs, Jobs in review, Completed Jobs, Pending payouts). Load data from the mock objects defined during setup.
2. **Quick actions** – Add buttons to navigate to `Add Worker`, `Create Job`, and `Review Submissions` pages.
3. **Job creation form** – Build a form with fields for job title, service category, description, expected output, deadline, pay amount and readiness level. On submit, update the mock job data array and display a success notification.
4. **Job list table** – Display jobs in a table with columns for title, category, assigned worker, status and actions. Implement sorting and filtering (status, category) by manipulating the mock data.

5. **Job details page** – Create a view to show job information, assignments and submission status. Include buttons to assign a worker and update status. Changes should update the in-memory mock data.

6. Worker Portal

1. **Profile page** – Allow workers to view and edit their profile (name, email, contact info). Show their training status and approved service categories. Store updates in local state or context.
2. **Training checklist** – Create a list of training tasks with checkboxes. Allow workers to mark tasks as done and save progress in local state (simulate saving by updating the mock data).
3. **Assigned jobs list** – Show all assigned jobs with key details and status badges. Provide an action to update progress (e.g., marking as “In Progress” or “Submitted”). Reflect changes in the mock job data.
4. **Job submission** – Build a submission form for uploading files and writing notes. Save submissions in mock data and update job status accordingly.
5. **Job history & payouts** – Display a history table of completed jobs with payout status. Link to details pages. Use static data or computed values from mock job entries.

7. Trainer Interface

1. **Trainee overview** – Show a list of trainees with their current training progress and job assignments.
2. **Review queue** – Present submissions from trainees. Provide controls to approve, request changes or comment. Update the status in the mock data accordingly.
3. **Trainer job management** – Trainers should have access to their own jobs just like workers. Extend worker pages with additional controls for trainer tasks.

8. Submission Review & Quality Control

1. **Review page** – Build a page to view job submissions. Display uploaded files/links in collapsible panels. Include a comments box for feedback and buttons for “Request changes”, “Approve”, “Reject”.
2. **Status updates** – Ensure that status changes (e.g., from “Submitted” to “Needs Revision” or “Approved”) trigger notifications and update the job entry.
3. **History log** – Record and display timestamps and notes for each status change.

9. Payout Tracking

1. **Payout table** – Show a list of completed jobs with worker name, pay amount, due date and payment status. Use dummy data from the mock data structure.
2. **Edit modal** – Create a modal to mark payouts as paid and add a reference note. Save updates to the mock data structure.

10. Polishing and Testing

1. **Add loading and error states** – Show spinners and error messages during API calls.
2. **Implement responsive styles** – Adjust layouts for tablet view. Ensure that cards, tables and forms adapt gracefully.
3. **Unit and integration tests** – Write simple tests using a testing library (e.g., Jest with React Testing Library) to ensure that components render correctly, state transitions occur as expected and

navigation flows work. Include tests for the role selection, job creation, job status changes and submission review. Codex should iterate until all tests pass.

4. **User testing** – Test the coded prototype internally and with the client. Fix any bugs or UX issues discovered.
5. **Refactoring** – Organize code into reusable components and hooks. Document the structure so it can be handed off for full production development.

11. Handover and Next Steps

1. **Deploy prototype** – Host the prototype on a staging site or local server for client review.
2. **Gather feedback** – Collect feedback on functionality, visuals and flows. Note changes for the production build.
3. **Plan production development** – Use insights from the prototype to refine the full project scope and timeline.

1 Plain Language—picking the right words keeps users coming and growing | IxDF

<https://ixdf.org/literature/article/plain-language-picking-the-right-words-keeps-users-coming-and-growing>